

```
(%o5) cos(%pi*m/2)
(%i6) declare(m, even); /* Will lead to simplification */
(%o6) done
(%i7) declare(n, odd);
(%o7) done
(%i8) cos(m * %pi);
(%o8) 1
(%i9) cos(n * %pi);
(%o9) - 1
(%i10) string(cos(m * %pi / 2));
(%o10) (-1)^(m/2)
(%i11) string(cos(n * %pi / 2));
(%o11) cos(%pi*n/2)
(%i12) quit();
```

Next is the relation between the normal and the hyperbolic trigonometric functions:

```
$ maxima -q
(%i1) sin(%i * x);
(%o1) %i sinh(x)
(%i2) cos(%i * x);
(%o2) cosh(x)
(%i3) tan(%i * x);
(%o3) %i tanh(x)
(%i4) quit();
```

By enabling the option variable *halfangles*, many half-angle identities come into play. To be specific, $\sin(x/2)$ gets further simplified in the $(0, 2 * \pi)$ range, and $\cos(x/2)$ gets further simplified in the $(-\pi/2, \pi/2)$ range. Check out the differences, before and after enabling the option variable, along with the range modifications, in the examples below:

```
$ maxima -q
(%i1) string(2*cos(x/2)^2 - 1); /* No effect */
(%o1) 2*cos(x/2)^2-1
(%i2) string(cos(x/2)); /* No effect */
(%o2) cos(x/2)
(%i3) halfangles:true; /* Enabling half angles */
(%o3) true
(%i4) string(2*cos(x/2)^2 - 1); /* Simplified */
(%o4) cos(x)
(%i5) string(cos(x/2)); /* Complex expansion for all x */
(%o5) (-1)^floor((x+%pi)/(2*%pi))*sqrt(cos(x)+1)/sqrt(2)
(%i6) assume(%pi < x, x < %pi); /* Limiting x values */
(%o6) [x > - %pi, x < %pi]
(%i7) string(cos(x/2)); /* Further simplified */
(%o7) sqrt(cos(x)+1)/sqrt(2)
(%i8) quit();
```

Trigonometric expansions and simplifications

Trigonometry is full of multiples of angles, the sums of angles, the products and the powers of trigonometric functions, and the long list of relations between them. Multiples and sums of angles fall into one category. The products and powers of trigonometric functions fall in another category. It's very useful to do conversions from one of these categories to the other one, to crack a range of simple and complex problems catering to a range of requirements—from basic hobby science to quantum mechanics. *trigexpand()* does the conversion from 'multiples and sums of angles' to 'products and powers of trigonometric functions'. *trigreduce()* does exactly the opposite. Here's a small demo:

```
$ maxima -q
(%i1) trigexpand(sin(2*x));
(%o1) 2 cos(x) sin(x)
(%i2) trigexpand(sin(x+y)-sin(x-y));
(%o2) 2 cos(x) sin(y)
(%i3) trigexpand(cos(2*x+y)-cos(2*x-y));
(%o3) - 2 sin(2 x) sin(y)
(%i4) trigexpand(%o3);
(%o4) - 4 cos(x) sin(x) sin(y)
(%i5) string(trigreduce(%o4));
(%o5) -2*(cos(y-2*x)/2-cos(y+2*x)/2)
(%i6) string(trigsimp(%o5));
(%o6) cos(y+2*x)-cos(y-2*x)
(%i7) string(trigexpand(cos(2*x)));
(%o7) cos(x)^2-sin(x)^2
(%i8) string(trigexpand(cos(2*x) + 2*sin(x)^2));
(%o8) sin(x)^2+cos(x)^2
(%i9) trigsimp(trigexpand(cos(2*x) + 2*sin(x)^2));
(%o9) 1
(%i10) quit();
```

In %o5 above, you might have noted that the 2s could have been cancelled for further simplification. But that is not the job of *trigreduce()*. For that we have to apply the *trigsimp()* function as shown in %i6. In fact, many other trigonometric identities-based simplifications are achieved using *trigsimp()*. Check out the %i7 to %o9 sequences for another such example. 

By: Anil Kumar Pugalia

The author is a hobbyist in open source hardware and software, with a passion for mathematics. A gold medallist from NIT Warangal and IISc Bangalore, mathematics and knowledge sharing are two of his many passions. Apart from that, he shares his experiments with Linux and embedded systems through his weekend workshops. Learn more about him and his experiments at <http://sysplay.in>. He can be reached at email@sarika-pugs.com.